

Limbaje Star-Free

Fișier de intrare `input.txt`
Fișier de ieșire `output.txt`

Mulțimea limbajelor Star-Free este o submulțime a limbajelor regulate. Un limbaj Star-Free poate fi construit inductiv pornind de la următoarele limbaje de bază:

- Mulțimea vidă: $\emptyset$, sau NIL.
- Orice simbol $a \in \Sigma$.

Putem construi alte limbaje Star-Free prin complementare, concatenare și reuniune:

- Dacă L este un limbaj Star-Free, atunci și complementul lui, $(\text{COMP } L)$ este Star-Free. O consecință imediată este că $(\text{COMP NIL}) = \Sigma^*$ este limbaj Star-Free.
- Dacă $L_1, L_2, \dots, L_k$ sunt limbaje Star-Free, atunci și concatenarea lor $(\text{CAT } L_1 L_2 \dots L_k)$ este Star-Free. Aici vom abuza de notație și vom considera că concatenarea între simboluri, rezultând într-un cuvânt, este de asemenea un limbaj de bază, e.g. putem primi ca input $a_1 a_2 \dots a_k$ și să înțelegem același lucru ca $(\text{CAT } a_1 a_2 \dots a_k)$.
- Dacă $L_1, L_2, \dots, L_k$ sunt limbaje Star-Free, atunci și reuniunea lor $(\text{UNION } L_1 L_2 \dots L_k)$ este Star-Free.

Dăm mai jos câteva exemple de limbaje Star-Free:

- Dacă $\Sigma = \{a, b, c\}$, atunci a^* este limbaj Star-Free: trebuie să construim un limbaj care nu acceptă niciun cuvânt care conține vreun b sau c : complementul lui $\Sigma^*(b|c)\Sigma^*$, sau $(\text{CAT } (\text{COMP NIL}) (\text{UNION } b c) (\text{COMP NIL}))$:
 $(\text{COMP } (\text{CAT } (\text{COMP NIL}) (\text{UNION } b c) (\text{COMP NIL})))$
- Dacă $\Sigma = \{a, b\}$, atunci $(ab)^*$ este limbaj Star-Free: trebuie să nu acceptăm cuvinte care încep cu b , se termină cu a , sau au două a sau b consecutive:
 $(\text{COMP } (\text{UNION } (\text{CAT } b (\text{COMP NIL})) (\text{CAT } (\text{COMP NIL}) a) (\text{CAT } (\text{COMP NIL}) aa (\text{COMP NIL})) (\text{CAT } (\text{COMP NIL}) bb (\text{COMP NIL}))))$
- Observați că nu putem folosi aceeași formulare pentru a arăta că $(aa)^*$ este limbaj Star-Free peste $\Sigma = \{a, b\}$, deoarece am putea accepta orice cuvânt de forma b^* .

Cerință

- Dându-se un limbaj Star-Free, determinați pentru mai multe cuvinte dacă acestea pot fi acceptate de limbaj (**MEMBERSHIP**).
- Dându-se două limbaje Star-Free, determinați dacă acestea sunt echivalente sau nu. Dacă răspunsul este negativ, trebuie să oferiți un exemplu care să demonstreze negația (**EQUIVALENCE**).

Date Intrare

- Pe prima linie este un singur cuvânt care determină tipul de interogare, **MEMBERSHIP** sau **EQUIVALENCE**.
- Pe a doua linie este un singur cuvânt, format din concatenarea tuturor simbolurilor din Σ .

Dacă interogarea este de tip **MEMBERSHIP**:

- Pe următoarea linie este descris un limbaj Star-Free L în formatul prezentat mai sus.
- Pe următoarea linie este un singur număr n , reprezentând numărul de cuvinte pentru care trebuie să determinăm dacă sunt acceptate de L .
- Pe următoarele n linii se află câte un cuvânt.

Dacă interogarea este de tip **EQUIVALENCE**:

- Pe următoarele două linii sunt descrise două limbaje Star-Free L_1, L_2 în formatul prezentat mai sus.

Date Ieșire

Dacă interogarea este de tip **MEMBERSHIP**, afișați n răspunsuri (**YES** sau **NO**), câte unul pe linie. Afișați **YES** pe a i -a linie dacă și numai dacă al i -lea cuvânt din input este acceptat de L .

Dacă interogarea este de tip **EQUIVALENCE**, afișați pe prima linie **YES** dacă cele două limbaje sunt echivalente sau **NO** altfel.

Dacă ați afișat **NO**, trebuie să afișați pe următoarea linie un cuvânt care să demonstreze că L_1, L_2 nu sunt echivalente. În plus, afișați pe a treia linie două verdicte **YES/NO** separate printr-un spațiu, reprezentând dacă L_1, L_2 acceptă sau nu cuvântul dat ca exemplu.

Restricții

- Σ este pentru toate testele o submulțime a celor 26 de caractere mici latine.
- Pentru grupele 1 și 2 putem garanta că pentru orice limbaj primit există un automat finit determinist care îl descrie și are cel mult 10 stări.
- Pentru grupele 3 și 4 putem garanta că pentru orice limbaj primit există un automat finit determinist care îl descrie și are cel mult 5000 de stări.

#	Punctaj	Restricții
1	30	testele din <code>tests.membership_small</code>
2	30	testele din <code>tests.equivalence_small</code>
3	30	testele din <code>tests.membership_big</code>
4	30	testele din <code>tests.equivalence_big</code>

Exemple

input.txt	output.txt
MEMBERSHIP ab (CAT (COMP NIL) a) 4 ba b aaa aab	YES NO YES NO
EQUIVALENCE a aa (CAT a (COMP (CAT a (COMP NIL)))) a)	YES
EQUIVALENCE ab (CAT (UNION (COMP (UNION (CAT a (COMP NIL))) (CAT b (COMP NIL)))) ab) ab) (UNION abab ababab)	NO ab YES NO

Explicații

- Pentru primul exemplu, trebuie să acceptăm un cuvânt dacă și numai dacă se termină în *a*.
- Pentru al doilea exemplu, primul limbaj acceptă doar cuvântul *aa*. Observați că partea (COMP (CAT a (COMP NIL))) din al doilea limbaj acceptă doar cuvântul gol.
- Pentru al treilea exemplu (liniile 3 și 4 sunt de fapt una singură), primul limbaj acceptă doar cuvintele *ab* și *abab*, astfel cele două cuvinte care pot fi folosite ca exemplu sunt *ab* și *ababab*.

Referințe utile

- Verificarea echivalenței în timp (aproape-) liniar a două DFA: [1]. Fun fact, articolul a fost scris în scurta perioadă în care era crezut că orice secvență de $n - 1$ uniri peste o structură Union-Find avea complexitate liniară [2].
- Minimizarea unui DFA în $O(|Q|^2)$ sau $O(|Q| \log |Q|)$: [3].
- Echivalența între două NFA: [4] (nu am implementat convertirea în NFA sau într-un hibrid DFA-NFA, posibil să fie utilă lucrarea pentru a obține întreg bonusul).

Precizări

- Checker-ul este disponibil pe gitlab.
- Tema va fi testată pe VMChecker. Limita de timp per test este de 120 secunde, iar pentru toată tema limita este de 540 de secunde. Nu există o limită de memorie pentru heap. Mașina virtuală are 4 GB de memorie alocată.
- **Makefile**-ul trebuie să se afle neapărat în rădăcina arhivei încărcate pe Moodle.
- Dimensiunea stivei este mărită de checker (doar în timpul rulării lui) la 256 MB. Dacă vă testați independent de checker programul, puteți să vă măriți dimensiunea stivei în sesiunea curentă de terminal cu `ulimit -s 262144`. Puteți să verificați dimensiunea stivei cu `ulimit -a | grep stack`.
- Checker-ul poate fi invocat astfel: `./checker checker_info.txt ../src/`, unde `checker_info.txt` este un fișier care descrie toate testele pe care va fi rulată sursa. Checker-ul va încerca să apeleze `make` din folder-ul dat ca argument (`../src/`), deci trebuie neapărat să existe un **Makefile** acolo cu regulile `build`, `run` și `clean`.
- Tentativele de a păcăli checker-ul vor fi pedepsite.
- Dacă vreți să implementați rezolvarea într-un limbaj care nu este suportat de imaginea de docker a checker-ului, postați un mesaj pe forum.